TransportTN

Application Mobile de Reservation de Transport

Taxi · Taxi Collectif · Bus Interurbain

CAHIER DES CHARGES — Version 1.0

Document confidentiel — Projet academique 2025

Android (Java) | Spring Boot | Firebase | MySQL 8

1. Présentation Générale du Projet

TransportTN est une application mobile Android destinée à moderniser et digitaliser le secteur des transports en commun en Tunisie. Elle permet aux citoyens de réserver un taxi privé, d'acheter une place dans un taxi collectif ou d'acquérir un ticket de bus interurbain, le tout depuis leur smartphone, avec confirmation instantanée et suivi GPS en temps réel.

1.1 Contexte et motivation

Le secteur du transport en Tunisie est encore largement informel : absence de réservation numérique, manque de visibilité sur les disponibilités, et difficultés pour les voyageurs de planifier leurs déplacements. TransportTN apporte une solution digitale complète, adaptée aux contraintes locales (paiement en espèces, réseau variable, langue française/arabe).

1.2 Objectifs du projet

- Créer une plateforme unifiée de réservation pour les trois modes de transport courants en Tunisie.
- Offrir une expérience utilisateur fluide, rapide et intuitive sur smartphone Android.
- Garantir la sécurité des transactions via JWT et l'authentification Firebase OTP.
- Fournir un suivi GPS en temps réel du véhicule pour les courses taxi.
- Générer des tickets QR infalsifiables pour les voyages en bus.
- Permettre une mise à l'échelle progressive vers d'autres villes et wilayas.

2. Périmètre Fonctionnel

2.1 Modules applicatifs

Module	Description	Priorité
Authentification	Connexion par OTP (numéro +216) via Firebase Auth	P1 — Essentiel
Réservation Taxi	Saisie départ/arrivée sur carte, estimation prix, matching chauffeur	P1 — Essentiel
Taxi Collectif	Sélection de ligne, choix du nombre de places, paiement	P1 — Essentiel
Tickets Bus	Sélection ligne/date, achat ticket, génération QR code	P1 — Essentiel
Suivi GPS temps réel	Position chauffeur mise à jour toutes les 3 secondes (Firestore)	P1 — Essentiel
Notifications Push	FCM : assignation chauffeur, statut course, annulation	P1 — Essentiel
Historique voyages	Liste des réservations passées avec statut et détails	P2 — Important
Gestion profil	Mise à jour des informations personnelles, token FCM	P2 — Important
Tableau de bord Admin	Gestion des lignes bus/collectif, stats réservations	P3 — Optionnel

2.2 Fonctionnalités détaillées — Réservation Taxi

- Saisie de l'adresse de départ et d'arrivée via Google Maps (géocodage inversé en français)

- Affichage de la carte avec marqueurs de départ/arrivée
- Calcul automatique de la distance (formule Haversine) et estimation du prix
- Choix du mode de paiement : espèces ou carte bancaire
- Algorithme de matching pour trouver le chauffeur disponible le plus proche
- Notification push au chauffeur et au client lors de l'assignation
- Suivi en temps réel de la position du chauffeur sur la carte (Firestore)
- Annulation possible avant la prise en charge

2.3 Fonctionnalités détaillées — Taxi Collectif

- Liste des lignes disponibles récupérée depuis l'API Spring Boot
- Affichage : itinéraire, heure de départ, prix par place, places restantes
- Sélection du nombre de places souhaitées (1 à 4)
- Vérification de la disponibilité en temps réel avant confirmation
- Confirmation par notification push

2.4 Fonctionnalités détaillées — Bus Interurbain

- Catalogue des lignes avec départ, destination, horaire et tarif
- Sélection de la date de voyage avec calendrier
- Choix de la quantité de tickets
- Génération d'un QR code unique après paiement (UUID base64)
- Stockage local et consultation des tickets depuis l'application

3. Architecture Technique

Le projet adopte une architecture trois-tiers classique enrichie de services cloud Firebase pour la gestion de l'authentification et du temps réel.

Couche	Technologie	Rôle
Frontend Mobile	Android Studio · Java · SDK 26+	Interface utilisateur, navigation, Maps
Backend REST	Spring Boot 3.2 · Java 17 · Maven	API REST, logique métier, JWT
Base de données	MySQL 8.0 · Spring Data JPA	Persistance des données métier
Authentification	Firebase Auth (Phone OTP)	Connexion sans mot de passe (+216)
Temps réel GPS	Firebase Firestore	Position chauffeur toutes les 3 secondes
Notifications	Firebase Cloud Messaging (FCM)	Push notifications Android
Réseau Android	Retrofit 2.9 · OkHttp 4.12	Appels API REST depuis l'app
Sécurité API	JJWT 0.12.3 · Spring Security 6	Tokens JWT, filtres d'autorisation

3.1 Schéma de base de données (6 tables)

Table	Colonnes principales	Description
users	id, phone, name, role, fcm_token, created_at	Clients et chauffeurs
drivers	id, user_id, vehicle_type, latitude, longitude, available	Profil chauffeur + position GPS
reservations	id, client_id, driver_id, type, status, pickup/dropoff, price	Réservations taxi/collectif
lignes	id, name, departure, destination, depart_time, price, total_seats	Lignes bus et collectif
tickets	id, client_id, ligne_id, travel_date, quantity, qr_code, status	Tickets bus avec QR
reservation_seats	id, reservation_id, ligne_id, seats_count, travel_date	Places taxi collectif

3.2 Sécurité

- Authentification sans mot de passe : OTP SMS via Firebase (numéros +216 uniquement)
- Chaque requête API est protégée par un token JWT signé (HMAC-SHA256, expiration 24h)
- Filtre Spring Security intercepte et valide le JWT avant chaque contrôleur
- Rôles distincts : CLIENT, DRIVER, ADMIN — accès aux endpoints filtrés par rôle
- CORS configuré pour n'accepter que les origines connues en production
- Fichier firebase-service-account.json exclu du contrôle de version (.gitignore)

4. Interfaces Utilisateur (UI/UX)

4.1 Parcours utilisateur principal

Étape	Écran	Action
1	Splash Screen	Vérification session existante
2	Saisie numéro	Entrer +216XXXXXXXX
3	Vérification OTP	Saisir le code SMS reçu
4	Accueil (Home)	Choisir : Taxi / Collectif / Bus
5a	Formulaire Taxi	Saisir départ, arrivée, mode paiement
5b	Liste Lignes	Parcourir et sélectionner une ligne
5c	Catalogue Bus	Choisir ligne, date, quantité
6	Confirmation	Récapitulatif et validation
7	Suivi / Ticket	Carte GPS (taxi) ou QR code (bus)

4.2 Exigences UI

- Design Material Design 3 (Google) avec thème couleurs rouge/blanc (drapeau tunisien)
- Navigation par BottomNavigationView (Accueil, Taxi, Collectif, Bus, Profil)
- Chargement asynchrone avec indicateurs ProgressBar pour toutes les requêtes réseau
- Messages d'erreur explicites et localisés en français
- Support des écrans 5" à 6.7" (densités hdpi, xhdpi, xxhdpi)
- Mode portrait uniquement pour garantir la cohérence visuelle

5. Spécifications de l'API REST

L'API REST est exposée par le backend Spring Boot sur le port 8080 (développement) et sur le port 443 via Nginx en production. Tous les endpoints (sauf /auth/login) nécessitent un header Authorization: Bearer .

Méthode	Endpoint	Description	Auth
POST	/api/auth/login	Firebase token → JWT applicatif	Non
GET	/api/auth/me	Profil de l'utilisateur connecté	JWT
PUT	/api/users/fcm-token	Enregistrer/mettre à jour le token FCM	JWT
POST	/api/reservations/taxi	Créer une réservation taxi	JWT
POST	/api/reservations/collectif	Réserver des places taxi collectif	JWT
POST	/api/reservations/bus	Acheter ticket(s) de bus	JWT
GET	/api/reservations/history	Historique des réservations client	JWT
PUT	/api/reservations/{id}/status	Mettre à jour le statut (chauffeur)	JWT
GET	/api/reservations/lignes	Lister les lignes disponibles	JWT
GET	/api/reservations/tickets	Mes tickets QR	JWT
PUT	/api/driver/location	Mettre à jour la position GPS	JWT (DRIVER)
PUT	/api/driver/available	Changer la disponibilité	JWT (DRIVER)
POST	/api/admin/lignes	Créer une nouvelle ligne	JWT (ADMIN)

■ *Toutes les réponses sont au format JSON. Les erreurs retournent un code HTTP approprié (400, 401, 403, 404, 500) accompagné d'un message descriptif.*

6. Contraintes Non Fonctionnelles

6.1 Performance

- Temps de réponse API < 500ms pour les requêtes de réservation (hors latence réseau)
- Mise à jour GPS toutes les 3 secondes via Firestore (streaming temps réel)
- Algorithme de matching chauffeur exécuté de manière asynchrone (non-bloquant)
- Pagination des listes (historique, lignes) pour limiter la charge serveur

6.2 Disponibilité et déploiement

- Disponibilité cible : 99,5% en production (SLA)
- Déploiement sur VPS Ubuntu 22.04 (minimum 2 Go RAM, 2 vCPU)
- HTTPS obligatoire en production via Nginx + Let's Encrypt (certificat SSL gratuit)
- Service systemd pour le redémarrage automatique du backend
- Support Docker Compose pour la reproductibilité des environnements
- CI/CD via GitHub Actions : tests automatiques + déploiement sur push vers main

6.3 Compatibilité

- Android SDK minimum : API 26 (Android 8.0 Oreo) — couvre 95%+ des appareils en Tunisie
- Android SDK cible : API 34 (Android 14)
- Compatible avec les appareils de 5" à 7" en mode portrait
- Testé sur émulateur Pixel 6 et appareils physiques Samsung/Huawei courants en Tunisie

6.4 Maintenabilité

- Architecture MVC côté Android avec séparation claire (Activity/Fragment/Service/Utils)
- Architecture en couches côté backend : Controller / Service / Repository / Model
- Couverture de tests unitaires cible : 80% pour la couche Service (JUnit 5 + Mockito)
- Tests instrumentés Espresso pour les flux critiques Android
- Documentation API via README complet et collection Postman (25 requêtes)

7. Planning et Livrables

7.1 Phases de développement

Phase	Contenu	Durée estimée
Phase 1 — Setup	Configuration projet Android, Spring Boot, Firebase, MySQL	1 semaine
Phase 2 — Auth	Firebase OTP Android + JWT Spring Boot	1 semaine
Phase 3 — Backend	Entités JPA, repositories, services métier, API REST	2 semaines
Phase 4 — Frontend	Fragments, Navigation, Google Maps, Retrofit	2 semaines
Phase 5 — Temps réel	FCM notifications, GPS Firestore, suivi carte	1 semaine
Phase 6 — Tests & Déploiement	JUnit, Espresso, Docker, CI/CD, Play Store	1 semaine

7.2 Livrables attendus

Livrable	Format	État
Code source backend (Spring Boot)	Dépôt GitHub / ZIP	Complété
Code source Android	Dépôt GitHub / ZIP	Complété
Collection Postman (25 requêtes)	JSON exportable	Complété
Schéma base de données	SQL + diagramme	Complété
Cahier des charges	PDF (ce document)	Complété
Présentation projet	PowerPoint 15 slides	En cours
AAB de release (Play Store)	app-release.aab	Planifié
Documentation README	Markdown GitHub	Complété

7.3 Idées pour la version 2.0

- Support de la langue arabe avec mise en page RTL (de droite à gauche)
- Intégration des solutions de paiement locales : Flouci et D17
- Système de notation mutuelle chauffeur / client (1 à 5 étoiles)
- Tableau de bord administrateur web (React ou Vue.js)
- Mode hors ligne avec synchronisation différée (Room database locale)
- Application chauffeur dédiée avec interface optimisée

8. Risques et Mitigations

Risque	Probabilité	Impact	Mitigation
Indisponibilité Firebase	Faible	Élevé	Circuit breaker + messages d'erreur utilisateur
Quota Google Maps dépassé	Moyenne	Moyen	Limiter les appels, utiliser le cache
Latence réseau mobile en Tunisie	Élevée	Moyen	Timeout Retrofit 30s, retry automatique
Refus Play Store (politique)	Faible	Élevé	Respecter les politiques dès la conception
Fuite token JWT	Faible	Élevé	Stockage SharedPreferences chiffré, expiration 24h
Performance serveur sous charge	Moyenne	Moyen	Mise à l'échelle verticale VPS, index MySQL

9. Conclusion

TransportTN est un projet complet et ambitieux qui répond à un besoin réel de digitalisation du transport en Tunisie. L'architecture choisie — Android natif, Spring Boot, Firebase, MySQL — est robuste, scalable et alignée avec les standards de l'industrie. L'ensemble des six phases de développement a été planifié pour couvrir le cycle complet : de la mise en place de l'environnement jusqu'au déploiement sur le Google Play Store.

Ce cahier des charges constitue la référence contractuelle et technique du projet. Il sera mis à jour au fil des évolutions pour refléter l'avancement réel du développement.

Réalisé par	Technologie	Année
Équipe TransportTN	Android · Spring Boot · Firebase	2025